Analyzing LLM extraction (distillation) attacks: latent clustering & latency-based anomaly detection

by Ahmed Elbanna

Inspired by OpenAI accusing DeepSeek of distilling its models, this work investigates the mechanics of Large Language Model (LLM) distillation attacks by analyzing how attackers systematically extract underlying reasoning capabilities from frontier models. By systematically analyzing API responses this notebook exposes extraction footprints such as predictable inter-query intervals and elevated reasoning marker frequencies. At the end it shows how timing-based defense mechanisms can expose automated harvesting.

In the GitHub repository you can find python code used to create and organise prompts data file divided into 5 categories: abstract (280), code_simple (280), commonsense (280), logic_chain (380), and math_basic (380) from gsm8k data and templates.

Then I used this example prompt data to collect the 1600 responses from gpt-40 (1200 prompts) and gpt-4o-mini (400 prompts) to create the responses data file responses_openai.csv and make this analysis.

In[1]:=
```
data = Import[
  "\\\\wsl.localhost\\Ubuntu-22.04\\home\\ahmedco31\\llm-distillation-analysis\\data\\
    responses_combined.csv", "Tabular"]
```

Out[1]=

	id	category	difficulty	source	prompt
40	abstract_022	abstract	medium	template_generated	Complete the pattern: A, E,
41	abstract_023	abstract	hard	template_generated	Identify the underlying patt
42	abstract_023	abstract	hard	template_generated	Identify the underlying patt
43	abstract_024	abstract	medium	template_generated	Which item does not belong
44	abstract_024	abstract	medium	template_generated	Which item does not belong
45	abstract_025	abstract	medium	template_generated	book is to read as music is t

Here is how many responses I collected from each model. API calls intervals were fixed for gpt-4o-

mini, while a uniform random between 10 and 25 for gpt-4o to reduce the collection time.

Out[7]=

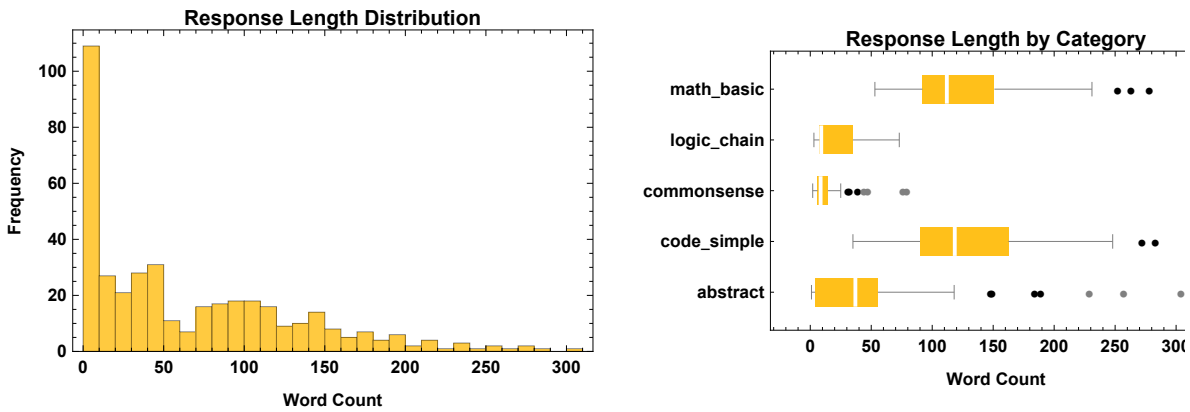
category	gpt–4o–mini	gpt–4o
abstract	80	200
code_simple	80	200
commonsense	80	200
logic_chain	80	300
math_basic	80	300

In[ ]:=
```
responses = Normal[data[All, "response"]];
wordCounts = Length[StringSplit[#]] & /@ responses;
```

Extract categories and group the word counts by them

```
hist = Histogram[wordCounts, ⋯];
groupedData =
  GroupBy[Transpose[{Normal[data[All, "category"]], wordCounts}], First → Last];
box = BoxWhiskerChart[Values[groupedData],
   {"Outliers", {"MedianMarker", 1, Directive[Thick, White]}}, ⋯];
GraphicsRow[{hist, box}, ⋯]
```

Out[ ]=



The heavily right-skewed distribution of response lengths is characteristic of a diverse query dataset aiming for broad domain coverage. Expected distillation behaviour. The boxplot shows how the word count is being distributed across the categories.

Reasoning pattern detection

Distilling a model's step-by-step reasoning process (Chain-of-Thought or CoT) is highly desirable because it allows an attacker to extract not just the final answer, but the underlying logic and problem-solving methodology of the target model. By tracking these markers, API providers can build anomaly detection systems to identify systematic queries designed to elicit these expensive reasoning chains, which strongly indicates a model extraction attack is occurring.

The bar chart below shows the average count of reasoning markers (like “first,” “then,” “therefore,” “step-by-step”) per response, grouped by category. High usage of reasoning markers in “math_basic”, “code_simple”, and “abstract” queries is shown.

Define markers and counting function

In[ ]:=
```
reasoningMarkers =
  {RegularExpression["\\b(first|second|third|finally|therefore|thus|hence)\\b"],
   RegularExpression["\\b(step \\d+|step-by-step)\\b"], RegularExpression[
    "\\b(because|since|given that)\\b"], RegularExpression["\\d+[\\.\\)]\\s"]};
countMarkers[text_String] :=
  Total[StringCount[text, reasoningMarkers, IgnoreCase → True]];
countMarkers[_] := 0;
```

Add the counts to the data

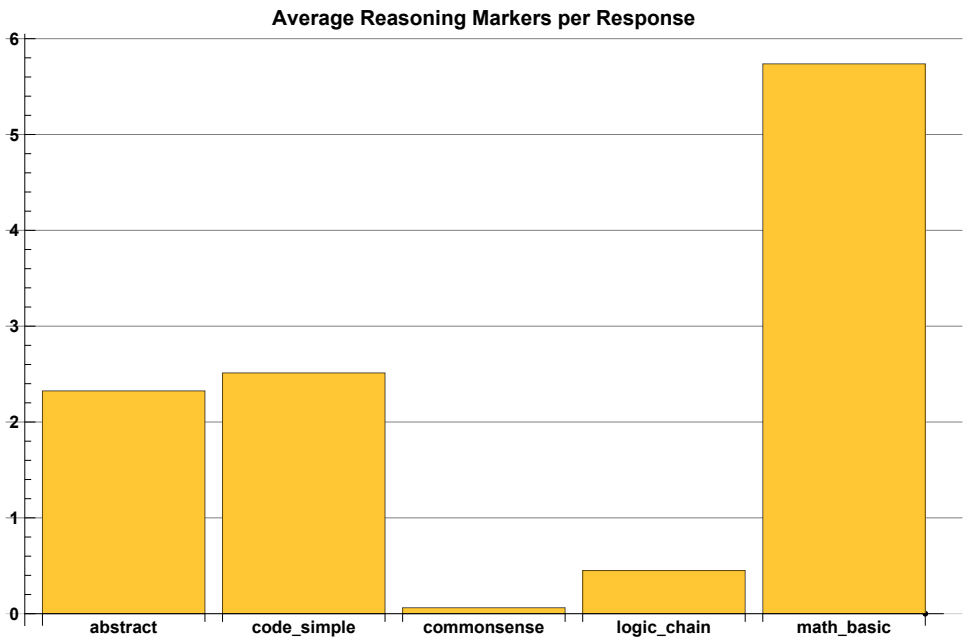
In[ ]:=
```
dataWithMarkers = InsertColumns[data,
   "reasoning_markers" → TabularColumn[countMarkers /@ Normal[data[All, "response"]]]];
Tabular[dataWithMarkers,
 Background → <|"ColumnRules" → {"reasoning_markers" → ■}|>, ImageSize → Medium]
```

Out[ ]=

	ɔt	tokens_completion	tokens_total	attempt	reasoning_markers
393	2	119	191	1	3
394	6	83	149	1	1
395	5	276	391	1	2
396	0	190	280	1	6
397	7	179	256	1	4
398	1	194	275	1	4
399	6	221	327	1	12
400	4	179	263	1	6

In[∘]:=
```
reasoningByCategory = Query[GroupBy["category"], <|"mean" → (Mean),
      "std" → (If[Length[#] > 1, StandardDeviation[#], 0.] &), "count" → Length|>,
    "reasoning_markers"][dataWithMarkers];
BarChart[Query[All, "mean"]@reasoningByCategory, ⋯]
```

Out[∘]=



In[∘]:=
```
Dataset@reasoningByCategory
```

Out[∘]=

	mean	std	count
abstract	2.325	4.64097	80
code_simple	2.5125	3.1462	80
commonsense	0.0625	0.459602	80
logic_chain	0.45	0.855363	80
math_basic	5.7375	3.23252	80

Query Pattern Analysis

I understand now why attackers add a randomly-lengthed intervals between queries, to mimic human behaviour when interacting with AI and avoid detection. When I made a fixed 20 seconds interval between queries, **the histogram of inter-query intervals gives me away immediately**. The variation in server response latency creates the skew we see in the histogram. The boxplot shows that the API's response latency is fundamentally tied to the category of the query, rather than the length of the query interval.

Would seeing the server processing time regularly spikes to 5000ms raises a red flag even though inter-query timing looks random?

Extract timestamps, convert to seconds since 1900, sort, and calculate differences

In[∘]:=
```
intervals = Differences@Sort@N[AbsoluteTime /@ Normal[data[All, "timestamp"]]];
```

In[ ]:=
```
medianInterval = Median@intervals;
```

Latencies and their mean

In[ ]:=
```
latencies = Normal[data[All, "latency_ms"]];
meanLatency = Mean[N@latencies];
```

Histogram vis.

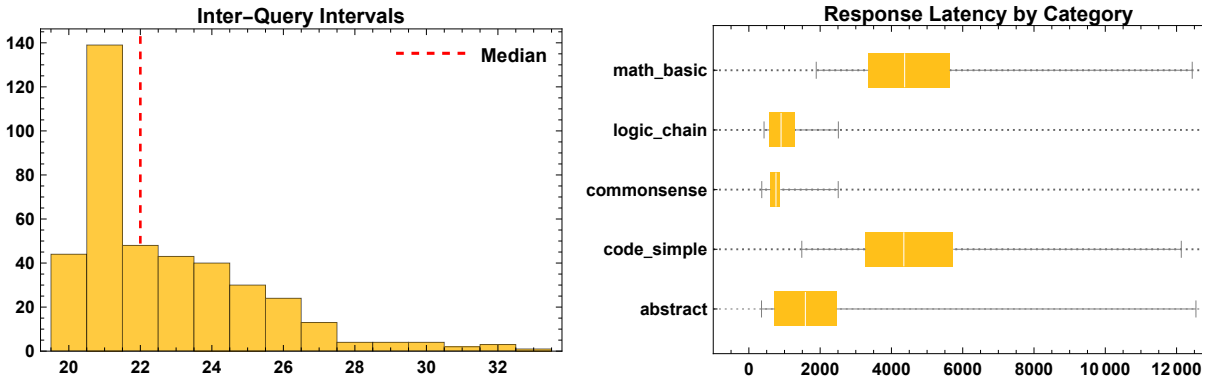
In[ ]:=
```
lHist = Histogram[intervals, ⋯ +];
```

Latency by category: extract categories and group the latencies by category

In[ ]:=
```
latencyByCategory =
  GroupBy[Transpose[{Normal[data[All, "category"]], latencies}], First → Last];
lBox = BoxWhiskerChart[Values[latencyByCategory], ⋯ +];
```

In[ ]:=
```
GraphicsRow[{lHist, lBox}, ImageSize → 660, Spacings → 10]
```

Out[ ]=



Prompt embeddings clustering

When you send prompts to a Large Language Model (LLM), it converts those text inputs into “embeddings”—massive lists of numbers (often thousands of dimensions long) that represent the mathematical “meaning” of the text.

As humans we cannot comprehend 1,000-dimensional space, we use UMAP (Uniform Manifold Approximation and Projection). UMAP is a dimensionality reduction algorithm that squashes those thousands of dimensions down to just two (the X and Y axes you see here). Crucially, UMAP preserves local structure: if two prompts mean similar things or trigger similar internal model behavior, they will fall in the same cluster.

Let’s start by extracting relevant values from the data

In[ ]:=
```
prompts = Normal[data[All, "prompt"]];
responses = Normal[data[All, "response"]];
categories = Normal[data[All, "category"]];
```

Then generate the embeddings. **FeatureExtract** handles state-of-the-art text embeddings natively, **DimensionReduce** supports UMAP. These embedding may not be exactly what different LLMs use, but they are close enough to study.

In[•]:=
```
promptEmbeddings = FeatureExtract[prompts, "SentenceVector"];
responseEmbeddings = FeatureExtract[responses, "SentenceVector"];
```

In[•]:=
```
Column[{Dimensions@promptEmbeddings, Dimensions@responseEmbeddings}]
```

Out[•]=
```
{400, 384}
{400, 384}
```

UMAP Reduction

In[•]:=
```
projections = DimensionReduce[promptEmbeddings, 2, Method → "UMAP"];
```

Creating tooltips

In[•]:=
```
plotPoints =
  Table[Tooltip[projections[[i]], Pane[Grid[{{Style["Prompt:", Bold], prompts[[i]]},
        {Style["Response:", Bold], responses[[i]]}}, Alignment → {Left, Top}],
      {400, Automatic} ]], {i, Length[prompts]}];
```

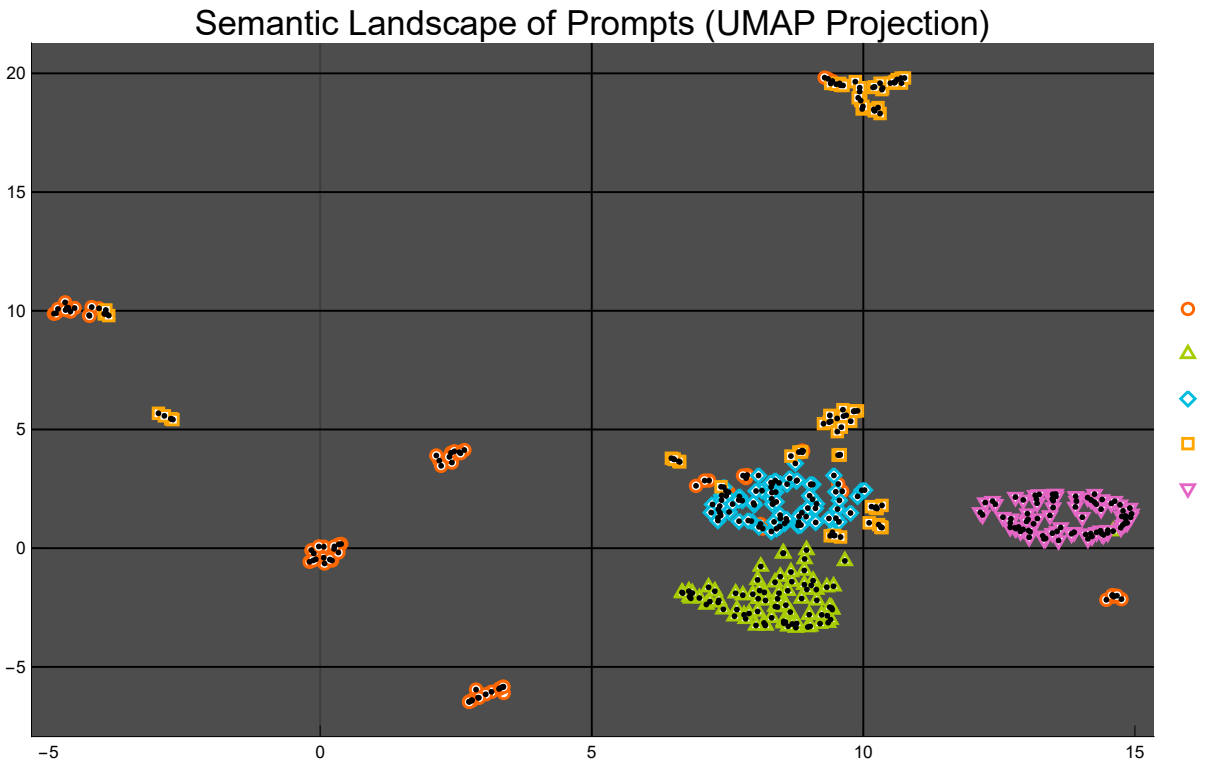
Group the points by category

In[•]:=
```
groupedData = GroupBy[Transpose[{categories, plotPoints}], First → Last];
```

The chart below proves that the LLM fundamentally treats these categories as entirely different computational tasks which forces the attackers to pick the categories they need knowledge for.

In[•]:=
```
ListPlot[Values[groupedData], ⋯ ]
```

Out[•]=



Look at the tight, completely isolated clusters for math_basic (magenta nablas), code_simple (green deltas), and knowledge (cyan rhombuses). The model isn't just generating text; it is routing these queries to highly specialized, distinct regions of its "brain" (its latent space).

Interestingly, the abstract prompts (orange circles) and logic_chain prompts (yellow squares) are

broken into multiple tiny sub-clusters scattered around the edges. This tells us that “abstract” isn’t a single behavior to the LLM; it triggers several different internal semantic pathways.

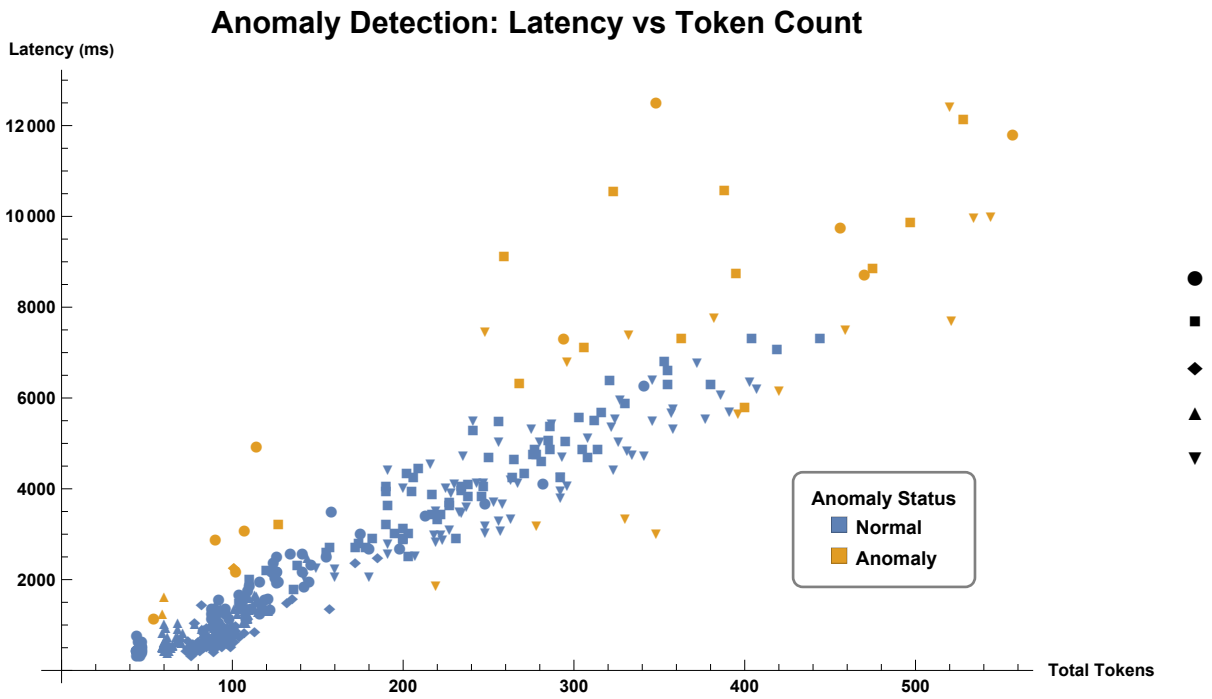
When a company (like DeepSeek) attempts to distill a frontier model (like OpenAI’s), they don’t just want random data. They need to systematically map and extract the entire semantic landscape. If an attacker only queried the green cluster (commonsense), their cloned student model would be completely incapable of writing code or doing math.

Infrastructure-level defense: timing-based anomaly detection

A latency vs tokens total chart with anomalies highlighted by Isolation Forest. Normally with prompt complexity getting higher, the latency gets higher, that is why we see the linear trend of the data. Having points outside the trend such as low token, high latency points in top left or high token, high latency in top right indicates unusual behaviour that worth checking. Attackers cannot easily fake the math of how GPUs generate tokens, making this a good defense tool against model extraction.

```
nativeBlue = ColorData[97, 1];
nativeRed = ColorData[97, 2];
anomalyColors = <|False → nativeBlue, True → nativeRed|>;
```

```
plots = Flatten@
   KeyValueMap[Function[{cat, anomalyGroup}, KeyValueMap[Function[{isAnom, points},
       ListPlot[points, PlotStyle → anomalyColors[isAnom], PlotMarkers →
         {categoryShapes[cat], 10}, PlotLegends → None]], anomalyGroup]], grouped];
```

```
Legended[Show[plots, ⋯ +],
 {Placed[PointLegend[ConstantArray[Black, 5], Keys[categoryShapes],
    LegendMarkers → (categoryShapes /@ Keys[categoryShapes]),
    LegendLabel → Style["Category", Bold]], Right],
  Placed[SwatchLegend[{nativeBlue, nativeRed}, {"Normal", "Anomaly"},
    LegendLabel → Style["Anomaly Status", Bold], LegendFunction → (Framed[#,
        Background → White, FrameStyle → Gray, RoundingRadius → 5] &)], {0.85, 0.25} ]}]
```

Out[•]=